TypeScript Advanced Concepts

Learning Objectives

- Master type guards and type narrowing techniques
- Work with discriminated unions for type safety
- Understand intersection types and mapped types
- Use conditional types for advanced type logic
- Apply utility types like Partial, Required, Pick, Omit
- Work with index signatures and type operators

Type Guards and Type Narrowing

Type guards are expressions that perform runtime checks to narrow down types. They help TypeScript understand what type a variable is within a specific code block.

typeof Type Guard

```
function printValue(value: string | number) {
  if (typeof value === "string") {
    // TypeScript knows value is string here
    console.log(value.toUpperCase());
  } else {
    // TypeScript knows value is number here
    console.log(value.toFixed(2));
  }
}

printValue("hello");
printValue(42.567);
```

Output:

```
HELLO
42.57
```

instanceof Type Guard

```
class Dog {
  bark() {
    console.log("Woof!");
  }
}

class Cat {
  meow() {
    console.log("Meow!");
  }
}

function makeSound(animal: Dog | Cat) {
  if (animal instanceof Dog) {
    animal.bark();
  } else {
    animal.meow();
  }
}

makeSound(new Dog());
makeSound(new Cat());
```

Output:

```
Woof!
Meow!
```

Custom Type Guard Functions

```
interface Fish {
  swim(): void;
}

interface Bird {
  fly(): void;
}

// Custom type guard
function isFish(animal: Fish | Bird): animal is Fish {
  return (animal as Fish).swim !== undefined;
}

function move(animal: Fish | Bird) {
  if (isFish(animal)) {
    animal.swim();
  } else {
    animal.fly();
```

```
  }
}

const fish: Fish = {
  swim: () => console.log("Swimming...")
};

const bird: Bird = {
  fly: () => console.log("Flying...")
};

move(fish);
move(bird);
```

Output:

```
Swimming...
Flying...
```

in Operator Type Guard

```
type Admin = {
  name: string;
  privileges: string[];
};

type User = {
  name: string;
  email: string;
};

function printInfo(person: Admin | User) {
  console.log(`Name: ${person.name}`);

  if ("privileges" in person) {
    console.log(`Privileges: ${person.privileges.join(", ")}`);
  }

  if ("email" in person) {
    console.log(`Email: ${person.email}`);
  }
}

printInfo({ name: "Alice", privileges: ["create", "delete"] });
printInfo({ name: "Bob", email: "bob@example.com" });
```

Output:

```
Name: Alice
Privileges: create, delete
Name: Bob
Email: bob@example.com
```

Discriminated Unions

Discriminated unions use a common property (discriminant) to distinguish between different types. This makes type narrowing safer and more predictable.

Basic Discriminated Union

```
type Circle = {
  kind: "circle";
  radius: number;
};

type Rectangle = {
  kind: "rectangle";
  width: number;
  height: number;
};

type Shape = Circle | Rectangle;

function getArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "rectangle":
      return shape.width * shape.height;
  }
}

const circle: Circle = { kind: "circle", radius: 5 };
const rectangle: Rectangle = { kind: "rectangle", width: 10, height: 20 };

console.log(getArea(circle).toFixed(2));
console.log(getArea(rectangle));
```

Output:

```
78.54
200
```

Discriminated Union for API Responses

```
type SuccessResponse = {
  status: "success";
  data: { id: number; name: string };
};

type ErrorResponse = {
  status: "error";
  error: string;
};

type ApiResponse = SuccessResponse | ErrorResponse;

function handleResponse(response: ApiResponse) {
  if (response.status === "success") {
    console.log(`Data: ${response.data.name}`);
  } else {
    console.log(`Error: ${response.error}`);
  }
}

handleResponse({ status: "success", data: { id: 1, name: "Alice" } });
handleResponse({ status: "error", error: "Not found" });
```

Output:

```
Data: Alice
Error: Not found
```

Intersection Types

Intersection types combine multiple types into one. The resulting type has all properties from all combined types.

Basic Intersection Type

```
type Person = {
  name: string;
  age: number;
};

type Employee = {
  employeeId: number;
  department: string;
};
```

```
type Staff = Person & Employee;

const staff: Staff = {
  name: "Alice",
  age: 30,
  employeeId: 12345,
  department: "Engineering"
};

console.log(`${staff.name} works in ${staff.department}`);
```

> Output:
>
> `Alice works in Engineering`

Intersection with Functions

```
type Printable = {
  print(): void;
};

type Saveable = {
  save(): void;
};

type Document = Printable & Saveable;

const doc: Document = {
  print: () => console.log("Printing document..."),
  save: () => console.log("Saving document...")
};

doc.print();
doc.save();
```

> Output:
>
> `Printing document...`
>
> `Saving document...`

Mapped Types

Mapped types transform properties of an existing type into a new type. They iterate over keys and apply transformations.

Making All Properties Optional

```
type User = {
  id: number;
  name: string;
  email: string;
};

type OptionalUser = {
  [K in keyof User]?: User[K];
};

const user1: OptionalUser = { id: 1 };
const user2: OptionalUser = { name: "Alice", email: "alice@example.com" };

console.log(user1);
console.log(user2);
```

Output:

```
{ id: 1 }
{ name: 'Alice', email: 'alice@example.com' }
```

Making All Properties Readonly

```
type Product = {
  id: number;
  name: string;
  price: number;
};

type ReadonlyProduct = {
  readonly [K in keyof Product]: Product[K];
};

const product: ReadonlyProduct = {
  id: 1,
  name: "Laptop",
  price: 999
};

console.log(product.name);
// product.price = 1200; // Error: Cannot assign to 'price'
```

Output:

```
Laptop
```

Conditional Types

Conditional types select one of two possible types based on a condition. They use the syntax `T extends U ? X : Y`.

Basic Conditional Type

```
type IsString = T extends string ? "yes" : "no";

type A = IsString;  // "yes"
type B = IsString;  // "no"

const result1: A = "yes";
const result2: B = "no";

console.log(result1);
console.log(result2);
```

> Output:
>
> `yes`
> `no`

Conditional Type with Function Return

```
type ArrayOrSingle = T extends any[] ? T[number] : T;

type Num = ArrayOrSingle;    // number
type Str = ArrayOrSingle;      // string

function getValue(value: number[]): Num;
function getValue(value: string): Str;
function getValue(value: any): any {
  return Array.isArray(value) ? value[0] : value;
}

console.log(getValue([10, 20, 30]));
console.log(getValue("hello"));
```

> Output:
>
> `10`
> `hello`

infer Keyword in Conditional Types

```
type ReturnType = T extends (...args: any[]) => infer R ? R : never;

function getUser() {
  return { id: 1, name: "Alice" };
}

function getCount() {
  return 42;
}

type UserReturnType = ReturnType;
type CountReturnType = ReturnType;

const user: UserReturnType = { id: 2, name: "Bob" };
const count: CountReturnType = 100;

console.log(user);
console.log(count);
```

> Output:
>
> `{ id: 2, name: 'Bob' }`
>
> `100`

Utility Types

TypeScript provides built-in utility types that transform existing types. These are powerful tools for type manipulation.

Partial<T>

Makes all properties of a type optional.

```
interface User {
  id: number;
  name: string;
  email: string;
}

function updateUser(id: number, updates: Partial) {
  console.log(`Updating user ${id} with:`, updates);
}
```

```
updateUser(1, { name: "Alice" });
updateUser(2, { email: "bob@example.com" });
```

> Output:
>
> `Updating user 1 with: { name: 'Alice' }`
>
> `Updating user 2 with: { email: 'bob@example.com' }`

Required<T>

Makes all properties of a type required.

```
interface Config {
  host?: string;
  port?: number;
  debug?: boolean;
}

type RequiredConfig = Required;

const config: RequiredConfig = {
  host: "localhost",
  port: 3000,
  debug: true
};

console.log(config);
```

> Output:
>
> `{ host: 'localhost', port: 3000, debug: true }`

Readonly<T>

Makes all properties of a type readonly.

```
interface Point {
  x: number;
  y: number;
}

const point: Readonly = { x: 10, y: 20 };
```

```
console.log(point);
// point.x = 15; // Error: Cannot assign to 'x'
```

Output:

```
{ x: 10, y: 20 }
```

Pick<T, K>

Creates a type by picking specific properties from another type.

```
interface Product {
  id: number;
  name: string;
  price: number;
  description: string;
}

type ProductPreview = Pick;

const preview: ProductPreview = {
  id: 1,
  name: "Laptop",
  price: 999
};

console.log(preview);
```

Output:

```
{ id: 1, name: 'Laptop', price: 999 }
```

Omit<T, K>

Creates a type by removing specific properties from another type.

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}

type PublicUser = Omit;
```

```
const publicUser: PublicUser = {
  id: 1,
  name: "Alice",
  email: "alice@example.com"
};

console.log(publicUser);
```

Output:

```
{ id: 1, name: 'Alice', email: 'alice@example.com' }
```

Record<K, T>

Creates an object type with specified keys and value types.

```
type Role = "admin" | "user" | "guest";

type Permissions = Record;

const permissions: Permissions = {
  admin: ["read", "write", "delete"],
  user: ["read", "write"],
  guest: ["read"]
};

console.log(permissions.admin);
console.log(permissions.guest);
```

Output:

```
[ 'read', 'write', 'delete' ]
[ 'read' ]
```

Index Signatures

Index signatures allow you to define types for objects with dynamic property names.

String Index Signature

```
interface StringDictionary {
  [key: string]: string;
}
```

```
const translations: StringDictionary = {
  hello: "hola",
  goodbye: "adiós",
  thanks: "gracias"
};

console.log(translations.hello);
console.log(translations["goodbye"]);
```

> Output:
>
> `hola`
>
> `adiós`

Number Index Signature

```
interface NumberArray {
  [index: number]: string;
}

const fruits: NumberArray = {
  0: "Apple",
  1: "Banana",
  2: "Orange"
};

console.log(fruits[0]);
console.log(fruits[2]);
```

> Output:
>
> `Apple`
>
> `Orange`

Mixed Index Signature with Known Properties

```
interface FlexibleConfig {
  name: string;
  version: number;
  [key: string]: string | number;
}

const config: FlexibleConfig = {
  name: "MyApp",
  version: 1,
```

```
  author: "Alice",
  port: 3000
};

console.log(config.name);
console.log(config.author);
console.log(config.port);
```

> Output:
>
> `MyApp`
>
> `Alice`
>
> `3000`

---

keyof Operator

The `keyof` operator creates a union type of all property names of a given type.

Basic keyof Usage

```
interface Person {
  name: string;
  age: number;
  email: string;
}

type PersonKeys = keyof Person;  // "name" | "age" | "email"

function getProperty(person: Person, key: PersonKeys) {
  return person[key];
}

const person: Person = {
  name: "Alice",
  age: 30,
  email: "alice@example.com"
};

console.log(getProperty(person, "name"));
console.log(getProperty(person, "age"));
```

> Output:

```
Alice
30
```

keyof with Generic Function

```
function pluck(items: T[], key: K): T[K][] {
  return items.map(item => item[key]);
}

const users = [
  { id: 1, name: "Alice", age: 30 },
  { id: 2, name: "Bob", age: 25 },
  { id: 3, name: "Charlie", age: 35 }
];

const names = pluck(users, "name");
const ages = pluck(users, "age");

console.log(names);
console.log(ages);
```

Output:

```
[ 'Alice', 'Bob', 'Charlie' ]
[ 30, 25, 35 ]
```

---

typeof Operator for Type Queries

The `typeof` operator can extract the type of a variable or property.

typeof with Variables

```
const user = {
  id: 1,
  name: "Alice",
  email: "alice@example.com"
};

type User = typeof user;

const newUser: User = {
  id: 2,
  name: "Bob",
  email: "bob@example.com"
```

```
};

console.log(newUser);
```

> Output:
>
> `{ id: 2, name: 'Bob', email: 'bob@example.com' }`

typeof with Functions

```
function createUser(name: string, age: number) {
  return { name, age, createdAt: new Date() };
}

type CreateUserReturn = ReturnType;

const user: CreateUserReturn = {
  name: "Alice",
  age: 30,
  createdAt: new Date()
};

console.log(user.name);
console.log(user.age);
```

> Output:
>
> `Alice`
>
> `30`

Practice Exercise

Create a generic function `filterByProperty` that filters an array of objects based on a property value. Use `keyof` to ensure type safety.

```
function filterByProperty(
  items: T[],
  key: K,
  value: T[K]
): T[] {
  return items.filter(item => item[key] === value);
}

interface Product {
```

```
  id: number;
  name: string;
  category: string;
  price: number;
}

const products: Product[] = [
  { id: 1, name: "Laptop", category: "Electronics", price: 999 },
  { id: 2, name: "Mouse", category: "Electronics", price: 25 },
  { id: 3, name: "Desk", category: "Furniture", price: 299 },
  { id: 4, name: "Chair", category: "Furniture", price: 199 }
];

const electronics = filterByProperty(products, "category", "Electronics");
const expensiveItems = filterByProperty(products, "price", 999);

console.log("Electronics:", electronics);
console.log("Price 999:", expensiveItems);
```

Output:

```
Electronics: [ { id: 1, name: 'Laptop', category: 'Electronics', price: 999 }, { id: 2, name: 'Mouse', category: 'Electronics', price: 25 } ]
Price 999: [ { id: 1, name: 'Laptop', category: 'Electronics', price: 999 } ]
```

Summary

In this module, you learned:

- Type guards (typeof, instanceof, custom) for type narrowing
- Discriminated unions for safe type discrimination
- Intersection types to combine multiple types
- Mapped types to transform existing types
- Conditional types for type-level logic
- Utility types: Partial, Required, Readonly, Pick, Omit, Record
- Index signatures for dynamic property names
- keyof operator for property name unions
- typeof operator for type queries

These advanced TypeScript concepts enable you to write highly type-safe and maintainable code. They are essential for building enterprise-level applications and working with complex type systems. Practice these patterns to master TypeScript's powerful type system.